In [1]:
```
import argparse
import os
import time
import shutil

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torch.backends.cudnn as cudnn


import torchvision
import torchvision.transforms as transforms

from models import *


global best_prec
use_gpu = torch.cuda.is_available()
print('=> Building model...')



batch_size = 128
model_name = "VGG16_4b_bottlenecked"
model = VGG_quant(vgg_name = 'VGG16_4b_bottlenecked', act_bit=4)

print(model)

normalize = transforms.Normalize(mean=[0.491, 0.482, 0.447], std=[0.247, 0.243


train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        normalize,
    ]))
trainloader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size


test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transforms.Compose([
        transforms.ToTensor(),
        normalize,
    ]))

testloader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size, s
```

```
print_freq = 100 # every 100 batches, accuracy printed. Here, each batch inclu
# CIFAR10 has 50,000 training data, and 10,000 validation data.

def train(trainloader, model, criterion, optimizer, epoch):
    batch_time = AverageMeter()
    data_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    model.train()

    end = time.time()
    for i, (input, target) in enumerate(trainloader):
        # measure data loading time
        data_time.update(time.time() - end)

        input, target = input.cuda(), target.cuda()

        # compute output
        output = model(input)
        loss = criterion(output, target)

        # measure accuracy and record loss
        prec = accuracy(output, target)[0]
        losses.update(loss.item(), input.size(0))
        top1.update(prec.item(), input.size(0))

        # compute gradient and do SGD step
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # measure elapsed time
        batch_time.update(time.time() - end)
        end = time.time()


        if i % print_freq == 0:
            print('Epoch: [{0}][{1}/{2}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Data {data_time.val:.3f} ({data_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   epoch, i, len(trainloader), batch_time=batch_time,
                   data_time=data_time, loss=losses, top1=top1))



def validate(val_loader, model, criterion ):
    batch_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    # switch to evaluate mode
    model.eval()

    end = time.time()
    with torch.no_grad():
        for i, (input, target) in enumerate(val_loader):
```

```
            input, target = input.cuda(), target.cuda()

            # compute output
            output = model(input)
            loss = criterion(output, target)

            # measure accuracy and record loss
            prec = accuracy(output, target)[0]
            losses.update(loss.item(), input.size(0))
            top1.update(prec.item(), input.size(0))

            # measure elapsed time
            batch_time.update(time.time() - end)
            end = time.time()

            if i % print_freq == 0:  # This line shows how frequently print ou
                print('Test: [{0}/{1}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   i, len(val_loader), batch_time=batch_time, loss=losses,
                   top1=top1))

    print(' * Prec {top1.avg:.3f}% '.format(top1=top1))
    return top1.avg


def accuracy(output, target, topk=(1,)):
    """Computes the precision@k for the specified values of k"""
    maxk = max(topk)
    batch_size = target.size(0)

    _, pred = output.topk(maxk, 1, True, True)
    pred = pred.t()
    correct = pred.eq(target.view(1, -1).expand_as(pred))

    res = []
    for k in topk:
        correct_k = correct[:k].view(-1).float().sum(0)
        res.append(correct_k.mul_(100.0 / batch_size))
    return res


class AverageMeter(object):
    """Computes and stores the average and current value"""
    def __init__(self):
        self.reset()

    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):
        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = self.sum / self.count
```

```
def save_checkpoint(state, is_best, fdir):
    filepath = os.path.join(fdir, 'checkpoint.pth')
    torch.save(state, filepath)
    if is_best:
        shutil.copyfile(filepath, os.path.join(fdir, 'model_best.pth.tar'))


def adjust_learning_rate(optimizer, epoch):
    """For resnet, the lr starts from 0.1, and is divided by 10 at 80 and 120 
    adjust_list = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140
    if epoch in adjust_list:
        for param_group in optimizer.param_groups:
            param_group['lr'] = param_group['lr'] * 0.9

#model = nn.DataParallel(model).cuda()
#all_params = checkpoint['state_dict']
#model.load_state_dict(all_params, strict=False)
#criterion = nn.CrossEntropyLoss().cuda()
#validate(testloader, model, criterion)
```

```
/Users/shanky/opt/anaconda3/envs/151AHW1/lib/python3.10/site-packages/tqdm/aut
o.py:22: TqdmWarning: IProgress not found. Please update jupyter and ipywidget
s. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
=> Building model...
VGG_quant(
  (features): Sequential(
    (0): QuantConv2d(
      3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_s
tats=True)
    (2): ReLU(inplace=True)
    (3): QuantConv2d(
      64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_s
tats=True)
    (5): ReLU(inplace=True)
    (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=F
alse)
    (7): QuantConv2d(
      64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (8): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
    (9): ReLU(inplace=True)
    (10): QuantConv2d(
      128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (11): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running
_stats=True)
    (12): ReLU(inplace=True)
    (13): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=
False)
    (14): QuantConv2d(
      128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (15): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running
_stats=True)
    (16): ReLU(inplace=True)
    (17): QuantConv2d(
      256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (18): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running
_stats=True)
    (19): ReLU(inplace=True)
    (20): QuantConv2d(
      256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (21): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running
_stats=True)
    (22): ReLU(inplace=True)
    (23): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=
False)
    (24): QuantConv2d(
      256, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
```

```
      (weight_quant): weight_quantize_fn()
    )
    (25): BatchNorm2d(8, eps=1e-05, momentum=0.1, affine=True, track_running_s
tats=True)
    (26): ReLU(inplace=True)
    (27): QuantConv2d(
      8, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (28): ReLU(inplace=True)
    (29): QuantConv2d(
      8, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (30): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running
_stats=True)
    (31): ReLU(inplace=True)
    (32): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=
False)
    (33): QuantConv2d(
      512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (34): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running
_stats=True)
    (35): ReLU(inplace=True)
    (36): QuantConv2d(
      512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (37): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running
_stats=True)
    (38): ReLU(inplace=True)
    (39): QuantConv2d(
      512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (40): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running
_stats=True)
    (41): ReLU(inplace=True)
    (42): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=
False)
    (43): AvgPool2d(kernel_size=1, stride=1, padding=0)
  )
  (classifier): Linear(in_features=512, out_features=10, bias=True)
)
Files already downloaded and verified
Files already downloaded and verified
```

In [ ]:

```
# This cell won't be given, but students will complete the training

lr = 0.01

weight_decay = 5e-4
momentum = 0.9
epochs = 200
best_prec = 0

#model = nn.DataParallel(model).cuda()
model.cuda()
```

```
criterion = nn.CrossEntropyLoss().cuda()
optimizer = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9, weight_de
#cudnn.benchmark = True

if not os.path.exists('result'):
    os.makedirs('result')
fdir = 'result/'+str(model_name)
if not os.path.exists(fdir):
    os.makedirs(fdir)


for epoch in range(0, epochs):
    adjust_learning_rate(optimizer, epoch)

    train(trainloader, model, criterion, optimizer, epoch)

    # evaluate on test set
    print("Validation starts")
    prec = validate(testloader, model, criterion)

    # remember best precision and save checkpoint
    is_best = prec > best_prec
    best_prec = max(prec,best_prec)
    print('best acc: {:1f}'.format(best_prec))
    save_checkpoint({
        'epoch': epoch + 1,
        'state_dict': model.state_dict(),
        'best_prec': best_prec,
        'optimizer': optimizer.state_dict(),
    }, is_best, fdir)
```

In [ ]:

```
# HW

#  1. Train with 4 bits for both weight and activation to achieve >90% accurac
#  2. Find x_int and w_int for the 2nd convolution layer
#  3. Check the recovered psum has similar value to the un-quantized original p
#     (such as example 1 in W3S2)
```

In [ ]:

```
PATH = "result/VGG16_quant/4b_best_pth.tar"
checkpoint = torch.load(PATH)
model.load_state_dict(checkpoint['state_dict'])
device = torch.device("cuda")

model.cuda()
model.eval()

test_loss = 0
correct = 0

with torch.no_grad():
    for data, target in testloader:
        data, target = data.to(device), target.to(device) # loading to GPU
        output = model(data)
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()

test_loss /= len(testloader.dataset)

print('\nTest set: Accuracy: {}/{} ({:.0f}%)\n'.format(
```

```
        correct, len(testloader.dataset),
        100. * correct / len(testloader.dataset)))
```

```
Test set: Accuracy: 9198/10000 (92%)
```

In [5]:
```
class SaveOutput:
    def __init__(self):
        self.outputs = []
    def __call__(self, module, module_in):
        self.outputs.append(module_in)
    def clear(self):
        self.outputs = []

save_output = SaveOutput()
```

In [6]:
```
#send an input and grap the value by using prehook like HW3
for idx, layer in enumerate(model.modules()):
    if isinstance(layer, QuantConv2d):
        print("prehooked:", idx)
        print(layer)
        layer.register_forward_pre_hook(save_output)

# we care about -5
```

```
prehooked: 2
QuantConv2d(
  3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
prehooked: 6
QuantConv2d(
  64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
prehooked: 11
QuantConv2d(
  64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
prehooked: 15
QuantConv2d(
  128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
prehooked: 20
QuantConv2d(
  128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
prehooked: 24
QuantConv2d(
  256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
prehooked: 28
QuantConv2d(
  256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
prehooked: 33
QuantConv2d(
  256, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
prehooked: 37
QuantConv2d(
  8, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
prehooked: 40
QuantConv2d(
  8, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
prehooked: 45
QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
prehooked: 49
QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
```

```
prehooked: 53
QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
```

In [7]:
```
dataiter = iter(testloader)
images, labels = next(dataiter)
images = images[0].unsqueeze(0)
output = model(images)
```

```
[W NNPACK.cpp:53] Could not initialize NNPACK! Reason: Unsupported hardware.
```

In [9]:
```
print(save_output.outputs[-5][0].shape)
print(save_output.outputs[-4][0].shape)
print(save_output.outputs[-5][0])
print(save_output.outputs[-4][0])
```

```
torch.Size([1, 8, 4, 4])
torch.Size([1, 8, 4, 4])
tensor([[[[0.1213, 0.0000, 0.5711, 0.3952],
          [0.0000, 0.0000, 0.0000, 0.0000],
          [0.0000, 0.0000, 0.0000, 0.0000],
          [0.0502, 0.0000, 0.2201, 0.0380]],

         [[0.0000, 0.0000, 0.0000, 0.0000],
          [0.0000, 0.0000, 0.0000, 0.0000],
          [0.0000, 0.0000, 0.0000, 0.0000],
          [0.0000, 0.0000, 0.0936, 0.0000]],

         [[0.2189, 0.0000, 0.0000, 0.0000],
          [0.2157, 0.0416, 0.0000, 0.0000],
          [0.1062, 0.0000, 0.1188, 0.0000],
          [0.1795, 0.0258, 0.0282, 0.0000]],

         [[0.1074, 0.2930, 0.5906, 0.3827],
          [0.0000, 0.0000, 0.0000, 0.0000],
          [0.0000, 0.0000, 0.0000, 0.0000],
          [0.0000, 0.0000, 0.0000, 0.0000]],

         [[0.1326, 0.1259, 0.4408, 0.1095],
          [0.3251, 0.4664, 0.7046, 0.2185],
          [0.2417, 0.2465, 0.1430, 0.0000],
          [0.0000, 0.0000, 0.0000, 0.0000]],

         [[0.1467, 0.0000, 0.0000, 0.0000],
          [0.0000, 0.0000, 0.0000, 0.0000],
          [0.0509, 0.0963, 0.4227, 0.2936],
          [0.0000, 0.1475, 0.1255, 0.0825]],

         [[0.1251, 0.1229, 0.0891, 0.1854],
          [0.0000, 0.0000, 0.0000, 0.0000],
          [0.0000, 0.0000, 0.0000, 0.0000],
          [0.1067, 0.1141, 0.1472, 0.0280]],

         [[0.0000, 0.2688, 0.0000, 0.1794],
          [0.3735, 1.1645, 0.3204, 0.3181],
          [0.5137, 1.1561, 0.5476, 0.3342],
          [0.0000, 0.3381, 0.0000, 0.0431]]]], grad_fn=<ReluBackward0>)
tensor([[[[1.0199, 0.5189, 2.4155, 0.0000],
          [3.2922, 3.2922, 4.2942, 1.1451],
          [3.5785, 4.9383, 6.0297, 2.8091],
          [1.4851, 3.0059, 2.6481, 0.3042]],

         [[1.1630, 3.0059, 3.8647, 2.5944],
          [1.7177, 2.7375, 2.5765, 0.8230],
          [0.0179, 0.5726, 0.2684, 0.1431],
          [0.0000, 0.0000, 0.0000, 0.0000]],

         [[0.0895, 3.0238, 1.7713, 1.6461],
          [1.4135, 3.4353, 0.1610, 0.0000],
          [1.8608, 3.5964, 1.2882, 0.3221],
          [0.6620, 2.1829, 1.1630, 0.6799]],

         [[0.0000, 0.0000, 0.0000, 0.0000],
          [0.0000, 0.0000, 0.0000, 0.0000],
          [0.0000, 0.0000, 0.0000, 0.1074],
          [0.0000, 0.0000, 0.0000, 0.0000]],
```

```
[[0.0000, 0.0000, 0.0000, 0.0000],
 [0.0000, 0.0000, 0.0000, 1.0914],
 [0.0000, 0.0000, 0.0000, 0.0000],
 [0.0000, 0.0000, 0.0000, 0.0000]],

[[0.6083, 1.9145, 4.1331, 1.0199],
 [0.6441, 2.0397, 2.2902, 1.2167],
 [0.0000, 0.0000, 0.0000, 0.0000],
 [0.0000, 0.0000, 0.0000, 0.0000]],

[[0.2147, 0.0000, 0.0000, 0.0000],
 [2.5407, 3.9900, 2.5407, 1.7534],
 [1.6998, 2.3797, 0.0000, 0.4115],
 [0.5547, 0.1074, 0.0000, 0.0000]],

[[0.0000, 0.0000, 0.0000, 0.1789],
 [0.0000, 0.0000, 0.0000, 0.0000],
 [1.6461, 2.0576, 2.4691, 1.7713],
 [1.4493, 1.7534, 1.1988, 0.4831]]]], grad_fn=<ReluBackward0>)
```

In [15]:

```
w_bit = 4
weight_q = model.features[27].weight_q # quantized value is stored during the 
w_alpha = model.features[27].weight_quant.wgt_alpha  # alpha is defined in you
w_delta = (w_alpha / (2**(w_bit-1)-1))    # delta can be calculated by using a
weight_int = weight_q / w_delta # w_int can be calculated by weight_q and w_de
print(weight_int) # you should see clean integer numbers
print(weight_int.shape)
```

```
tensor([[[[-7.0000, -7.0000, -7.0000],
          [-7.0000,  5.0000,  7.0000],
          [ 7.0000,  1.0000, -3.0000]],

         [[ 0.0000, -7.0000, -7.0000],
          [ 7.0000,  2.0000,  7.0000],
          [ 0.0000,  3.0000,  0.0000]],

         [[ 1.0000,  4.0000, -7.0000],
          [-2.0000, -4.0000, -5.0000],
          [-1.0000, -2.0000, -7.0000]],

         [[-2.0000, -2.0000,  7.0000],
          [-7.0000, -7.0000, -3.0000],
          [-3.0000, -1.0000, -7.0000]],

         [[ 7.0000,  7.0000, -2.0000],
          [ 3.0000,  7.0000, -0.0000],
          [-7.0000,  5.0000, -2.0000]],

         [[-5.0000, -4.0000,  4.0000],
          [-2.0000,  1.0000,  4.0000],
          [ 7.0000,  7.0000,  7.0000]],

         [[-7.0000, -7.0000, -2.0000],
          [ 2.0000,  3.0000,  7.0000],
          [ 4.0000,  4.0000,  2.0000]],

         [[ 7.0000,  4.0000,  2.0000],
          [ 3.0000,  6.0000,  7.0000],
          [ 7.0000, -2.0000,  4.0000]]],


        [[[-1.0000,  6.0000,  7.0000],
          [ 2.0000,  7.0000,  7.0000],
          [-6.0000, -4.0000,  2.0000]],

         [[ 2.0000,  4.0000, -5.0000],
          [-7.0000,  7.0000, -6.0000],
          [-7.0000,  2.0000, -2.0000]],

         [[-1.0000,  2.0000, -3.0000],
          [ 5.0000, -0.0000,  1.0000],
          [-1.0000, -5.0000, -7.0000]],

         [[-2.0000,  0.0000,  0.0000],
          [ 7.0000,  2.0000, -3.0000],
          [-0.0000, -7.0000, -7.0000]],

         [[ 5.0000,  6.0000,  7.0000],
          [ 5.0000,  2.0000,  7.0000],
          [ 4.0000,  2.0000,  5.0000]],

         [[ 4.0000,  7.0000,  1.0000],
          [ 1.0000,  7.0000,  7.0000],
          [-7.0000, -7.0000, -7.0000]],

         [[-7.0000,  7.0000,  7.0000],
          [-7.0000, -2.0000,  5.0000],
          [-7.0000, -7.0000, -7.0000]],
```

```
 [[-7.0000, -3.0000, -5.0000],
  [ 1.0000, -3.0000,  2.0000],
  [ 4.0000,  2.0000,  1.0000]]],


[[[-0.0000,  2.0000,  5.0000],
  [ 7.0000,  5.0000,  7.0000],
  [-1.0000, -5.0000, -7.0000]],

 [[ 7.0000, -6.0000, -7.0000],
  [ 4.0000, -7.0000, -7.0000],
  [ 7.0000, -0.0000,  1.0000]],

 [[-1.0000,  2.0000,  7.0000],
  [-1.0000, -2.0000, -0.0000],
  [-4.0000, -5.0000, -7.0000]],

 [[-7.0000, -7.0000, -2.0000],
  [ 7.0000,  2.0000,  7.0000],
  [ 7.0000,  7.0000,  7.0000]],

 [[ 2.0000, -1.0000,  3.0000],
  [-5.0000,  4.0000,  7.0000],
  [-5.0000, -1.0000,  0.0000]],

 [[ 0.0000, -3.0000,  0.0000],
  [-7.0000, -4.0000,  0.0000],
  [-7.0000, -3.0000,  3.0000]],

 [[ 2.0000,  2.0000,  0.0000],
  [-2.0000,  3.0000, -1.0000],
  [-2.0000, -1.0000, -1.0000]],

 [[ 4.0000,  7.0000, -1.0000],
  [ 1.0000,  7.0000,  4.0000],
  [-2.0000,  7.0000, -5.0000]]],


[[[-3.0000,  2.0000, -4.0000],
  [-1.0000, -1.0000, -7.0000],
  [ 7.0000, -7.0000, -7.0000]],

 [[-1.0000, -3.0000, -3.0000],
  [ 7.0000,  2.0000, -1.0000],
  [-4.0000, -0.0000, -0.0000]],

 [[-1.0000,  7.0000,  7.0000],
  [-2.0000,  7.0000,  7.0000],
  [-7.0000, -7.0000,  1.0000]],

 [[-1.0000,  1.0000,  1.0000],
  [-6.0000,  1.0000,  7.0000],
  [ 0.0000, -4.0000, -5.0000]],

 [[ 7.0000,  7.0000, -5.0000],
  [ 7.0000,  1.0000,  7.0000],
  [-7.0000, -3.0000, -2.0000]],

 [[-3.0000, -4.0000,  7.0000],
```

```
  [ 1.0000,  5.0000,  7.0000],
  [-4.0000,  2.0000,  7.0000]],

 [[ 3.0000,  1.0000, -7.0000],
  [ 7.0000,  4.0000,  2.0000],
  [-3.0000,  3.0000, -7.0000]],

 [[-7.0000, -7.0000, -1.0000],
  [-7.0000, -3.0000, -1.0000],
  [-7.0000, -2.0000, -7.0000]]],


[[[ 7.0000,  7.0000,  7.0000],
  [ 7.0000,  7.0000,  7.0000],
  [-7.0000, -0.0000, -1.0000]],

 [[ 1.0000,  1.0000,  4.0000],
  [ 7.0000,  2.0000,  4.0000],
  [-1.0000, -1.0000, -2.0000]],

 [[-5.0000,  7.0000,  7.0000],
  [ 6.0000,  4.0000,  4.0000],
  [-2.0000, -7.0000, -7.0000]],

 [[ 4.0000,  1.0000,  1.0000],
  [-3.0000,  0.0000,  4.0000],
  [-7.0000, -3.0000,  6.0000]],

 [[-2.0000,  7.0000,  7.0000],
  [-3.0000, -2.0000, -7.0000],
  [-6.0000, -7.0000, -7.0000]],

 [[-2.0000,  3.0000, -1.0000],
  [ 5.0000, -7.0000, -7.0000],
  [ 7.0000,  1.0000, -7.0000]],

 [[-7.0000, -2.0000, -7.0000],
  [-1.0000, -2.0000, -7.0000],
  [-4.0000,  1.0000,  4.0000]],

 [[-4.0000,  1.0000, -3.0000],
  [-5.0000, -0.0000, -7.0000],
  [-5.0000,  5.0000, -3.0000]]],


[[[-4.0000, -5.0000,  2.0000],
  [-7.0000,  3.0000, -1.0000],
  [ 7.0000,  4.0000,  5.0000]],

 [[ 7.0000,  7.0000,  7.0000],
  [-0.0000,  7.0000,  7.0000],
  [-7.0000, -3.0000,  1.0000]],

 [[-2.0000, -5.0000, -2.0000],
  [-1.0000, -1.0000, -1.0000],
  [-2.0000, -2.0000, -2.0000]],

 [[ 7.0000, -2.0000,  7.0000],
  [ 7.0000,  5.0000,  6.0000],
  [ 7.0000, -7.0000, -0.0000]],
```

```
 [[ 5.0000,  3.0000, -7.0000],
  [ 2.0000,  7.0000,  7.0000],
  [-2.0000,  7.0000,  7.0000]],

 [[ 7.0000, -4.0000,  7.0000],
  [-7.0000, -7.0000, -3.0000],
  [ 4.0000, -2.0000,  1.0000]],

 [[-2.0000,  2.0000, -1.0000],
  [ 4.0000,  3.0000,  0.0000],
  [ 3.0000, -2.0000, -7.0000]],

 [[-7.0000, -0.0000, -5.0000],
  [ 3.0000, -0.0000,  4.0000],
  [ 5.0000, -5.0000, -4.0000]]],


[[[ 0.0000,  7.0000,  0.0000],
  [-7.0000, -3.0000, -5.0000],
  [-7.0000, -1.0000, -2.0000]],

 [[ 7.0000,  0.0000, -0.0000],
  [-1.0000, -1.0000, -4.0000],
  [ 7.0000,  2.0000, -0.0000]],

 [[ 7.0000, -7.0000, -7.0000],
  [ 7.0000,  7.0000,  1.0000],
  [ 7.0000,  7.0000,  4.0000]],

 [[-7.0000, -2.0000,  7.0000],
  [-5.0000,  1.0000, -5.0000],
  [ 0.0000,  7.0000, -0.0000]],

 [[ 7.0000,  7.0000, -4.0000],
  [ 5.0000,  7.0000, -0.0000],
  [-4.0000, -1.0000, -7.0000]],

 [[-3.0000, -5.0000, -7.0000],
  [-4.0000, -5.0000, -7.0000],
  [-1.0000, -3.0000, -2.0000]],

 [[ 1.0000,  7.0000,  7.0000],
  [ 1.0000,  7.0000,  3.0000],
  [ 4.0000,  1.0000, -7.0000]],

 [[-7.0000,  4.0000, -1.0000],
  [-4.0000,  5.0000,  6.0000],
  [ 1.0000,  5.0000, -1.0000]]],


[[[-7.0000, -4.0000, -1.0000],
  [-7.0000, -1.0000,  7.0000],
  [-7.0000,  1.0000,  4.0000]],

 [[ 7.0000,  7.0000,  7.0000],
  [-0.0000, -2.0000,  1.0000],
  [-5.0000, -7.0000,  7.0000]],

 [[-2.0000, -1.0000, -0.0000],
```

```
          [-0.0000,  2.0000,  7.0000],
          [ 7.0000,  4.0000,  7.0000]],

         [[-4.0000, -1.0000,  7.0000],
          [ 7.0000, -4.0000, -4.0000],
          [ 7.0000,  1.0000, -7.0000]],

         [[ 7.0000, -2.0000, -7.0000],
          [ 7.0000, -1.0000, -7.0000],
          [ 3.0000, -3.0000, -7.0000]],

         [[ 6.0000, -2.0000,  1.0000],
          [ 5.0000,  7.0000,  2.0000],
          [ 2.0000, -5.0000, -4.0000]],

         [[-7.0000, -3.0000, -2.0000],
          [-7.0000,  1.0000,  3.0000],
          [ 2.0000,  7.0000,  1.0000]],

         [[ 2.0000,  1.0000,  7.0000],
          [ 1.0000,  4.0000,  3.0000],
          [-2.0000, -3.0000, -2.0000]]]], grad_fn=<DivBackward0>)
torch.Size([8, 8, 3, 3])
```

In [11]:

```
act = save_output.outputs[-5][0]
act_alpha  = model.features[27].act_alpha
act_bit = 4
act_quant_fn = act_quantization(act_bit)

act_q = act_quant_fn(act, act_alpha)

act_int = act_q / (act_alpha / (2**act_bit-1))
print(act_int)
```

```
tensor([[[[ 1.0000,  0.0000,  6.0000,  4.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  2.0000,  0.0000]],

         [[ 0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  1.0000,  0.0000]],

         [[ 2.0000,  0.0000,  0.0000,  0.0000],
          [ 2.0000,  0.0000,  0.0000,  0.0000],
          [ 1.0000,  0.0000,  1.0000,  0.0000],
          [ 2.0000,  0.0000,  0.0000,  0.0000]],

         [[ 1.0000,  3.0000,  6.0000,  4.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000]],

         [[ 1.0000,  1.0000,  4.0000,  1.0000],
          [ 3.0000,  5.0000,  7.0000,  2.0000],
          [ 2.0000,  2.0000,  1.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000]],

         [[ 1.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000],
          [ 1.0000,  1.0000,  4.0000,  3.0000],
          [ 0.0000,  1.0000,  1.0000,  1.0000]],

         [[ 1.0000,  1.0000,  1.0000,  2.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000],
          [ 1.0000,  1.0000,  1.0000,  0.0000]],

         [[ 0.0000,  3.0000,  0.0000,  2.0000],
          [ 4.0000, 11.0000,  3.0000,  3.0000],
          [ 5.0000, 11.0000,  5.0000,  3.0000],
          [ 0.0000,  3.0000,  0.0000,  0.0000]]]], grad_fn=<DivBackward0>)
```

In [18]:

```
conv_int = torch.nn.Conv2d(in_channels = 8, out_channels=8, kernel_size = 3, pa
conv_int.weight = torch.nn.parameter.Parameter(weight_int)
conv_int.bias = model.features[27].bias
output_int = conv_int(act_int)
print(output_int)
output_recovered = output_int * (act_alpha / (2**act_bit-1)) * (w_alpha / (2**
print(output_recovered)
```

```
tensor([[[[  57.0000,   29.0000,  135.0000,  -77.0000],
          [ 184.0000,  184.0000,  240.0000,   64.0000],
          [ 200.0000,  276.0000,  337.0000,  157.0000],
          [  83.0000,  168.0000,  148.0000,   17.0000]],

         [[  65.0000,  168.0000,  216.0000,  145.0000],
          [  96.0000,  153.0000,  144.0000,   46.0000],
          [   1.0000,   32.0000,   15.0000,    8.0000],
          [ -18.0000,  -34.0000,  -24.0000,   -5.0000]],

         [[   5.0000,  169.0000,   99.0000,   92.0000],
          [  79.0000,  192.0000,    9.0000,  -73.0000],
          [ 104.0000,  201.0000,   72.0000,   18.0000],
          [  37.0000,  122.0000,   65.0000,   38.0000]],

         [[ -67.0000, -102.0000, -172.0000,  -86.0000],
          [ -57.0000, -141.0000, -140.0000,  -30.0000],
          [ -52.0000, -132.0000,  -88.0000,    6.0000],
          [  -8.0000,  -70.0000,  -82.0000,  -56.0000]],

         [[-102.0000,  -36.0000, -129.0000,  -23.0000],
          [-135.0000,  -74.0000, -110.0000,   61.0000],
          [ -89.0000,  -41.0000, -139.0000,  -43.0000],
          [ -20.0000,   -2.0000,  -42.0000,   -5.0000]],

         [[  34.0000,  107.0000,  231.0000,   57.0000],
          [  36.0000,  114.0000,  128.0000,   68.0000],
          [ -54.0000,  -49.0000,  -25.0000,   -1.0000],
          [ -55.0000,  -24.0000,  -60.0000,  -40.0000]],

         [[  12.0000,  -17.0000,  -38.0000,  -40.0000],
          [ 142.0000,  223.0000,  142.0000,   98.0000],
          [  95.0000,  133.0000,  -12.0000,   23.0000],
          [  31.0000,    6.0000, -117.0000,  -59.0000]],

         [[ -71.0000,  -90.0000,  -49.0000,   10.0000],
          [ -28.0000,  -49.0000,  -40.0000,  -25.0000],
          [  92.0000,  115.0000,  138.0000,   99.0000],
          [  81.0000,   98.0000,   67.0000,   27.0000]]]],
       grad_fn=<ConvolutionBackward0>)
tensor([[[[ 1.0199,  0.5189,  2.4155, -1.3777],
          [ 3.2922,  3.2922,  4.2942,  1.1451],
          [ 3.5785,  4.9383,  6.0297,  2.8091],
          [ 1.4851,  3.0059,  2.6481,  0.3042]],

         [[ 1.1630,  3.0059,  3.8647,  2.5944],
          [ 1.7177,  2.7375,  2.5765,  0.8230],
          [ 0.0179,  0.5726,  0.2684,  0.1431],
          [-0.3221, -0.6083, -0.4294, -0.0895]],

         [[ 0.0895,  3.0238,  1.7713,  1.6461],
          [ 1.4135,  3.4353,  0.1610, -1.3061],
          [ 1.8608,  3.5964,  1.2882,  0.3221],
          [ 0.6620,  2.1829,  1.1630,  0.6799]],

         [[-1.1988, -1.8250, -3.0775, -1.5387],
          [-1.0199, -2.5228, -2.5049, -0.5368],
          [-0.9304, -2.3618, -1.5745,  0.1074],
          [-0.1431, -1.2525, -1.4672, -1.0020]],
```

```
          [[-1.8250, -0.6441, -2.3081, -0.4115],
           [-2.4155, -1.3240, -1.9682,  1.0914],
           [-1.5924, -0.7336, -2.4870, -0.7694],
           [-0.3578, -0.0358, -0.7515, -0.0895]],

          [[ 0.6083,  1.9145,  4.1331,  1.0199],
           [ 0.6441,  2.0397,  2.2902,  1.2167],
           [-0.9662, -0.8767, -0.4473, -0.0179],
           [-0.9841, -0.4294, -1.0735, -0.7157]],

          [[ 0.2147, -0.3042, -0.6799, -0.7157],
           [ 2.5407,  3.9900,  2.5407,  1.7534],
           [ 1.6998,  2.3797, -0.2147,  0.4115],
           [ 0.5547,  0.1074, -2.0934, -1.0556]],

          [[-1.2704, -1.6103, -0.8767,  0.1789],
           [-0.5010, -0.8767, -0.7157, -0.4473],
           [ 1.6461,  2.0576,  2.4691,  1.7713],
           [ 1.4493,  1.7534,  1.1988,  0.4831]]]], grad_fn=<MulBackward0>)
```

In [20]:

```
## This cell is provided

conv_ref = torch.nn.Conv2d(in_channels = 8, out_channels=8, kernel_size = 3, pa
conv_ref.weight = model.features[27].weight_q
conv_ref.bias = model.features[27].bias
output_ref = conv_ref(act)
#print(output_ref)

print(abs((output_ref - output_recovered)).mean())
```

```
tensor(0.1096, grad_fn=<MeanBackward0>)
```

In [45]:

```
act_int_single = act_int[0, ...] # take only first of the batch
C, H, W = act_int_single.shape
p = 1

a_pad = torch.zeros((C, H + 2 * p, W + 2 * p))
a_pad[:, p:p+H, p:p+W] = act_int_single

a_pad_flat = torch.reshape(a_pad, (a_pad.size(0), -1))
a_pad_nij = torch.permute(a_pad_flat, (1,0))

print(act_int_single)
print(a_pad)

print(a_pad.shape)

print(a_pad_nij)
print(a_pad_nij.shape)

w_int_kij = torch.reshape(weight_int, (weight_int.size(0), weight_int.size(1),
print(w_int_kij)
print(w_int_kij.shape)

output_int_single = output_int[0, ...]
out_int_flat = torch.reshape(output_int_single, (output_int_single.size(0), -1
out_int_nij = torch.permute(out_int_flat, (1, 0))
# apply reLU
out_int_nij.clamp_(min=0)
```

```
tensor([[[ 1.0000,  0.0000,  6.0000,  4.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  2.0000,  0.0000]],

        [[ 0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  1.0000,  0.0000]],

        [[ 2.0000,  0.0000,  0.0000,  0.0000],
         [ 2.0000,  0.0000,  0.0000,  0.0000],
         [ 1.0000,  0.0000,  1.0000,  0.0000],
         [ 2.0000,  0.0000,  0.0000,  0.0000]],

        [[ 1.0000,  3.0000,  6.0000,  4.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000]],

        [[ 1.0000,  1.0000,  4.0000,  1.0000],
         [ 3.0000,  5.0000,  7.0000,  2.0000],
         [ 2.0000,  2.0000,  1.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000]],

        [[ 1.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000],
         [ 1.0000,  1.0000,  4.0000,  3.0000],
         [ 0.0000,  1.0000,  1.0000,  1.0000]],

        [[ 1.0000,  1.0000,  1.0000,  2.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000],
         [ 1.0000,  1.0000,  1.0000,  0.0000]],

        [[ 0.0000,  3.0000,  0.0000,  2.0000],
         [ 4.0000, 11.0000,  3.0000,  3.0000],
         [ 5.0000, 11.0000,  5.0000,  3.0000],
         [ 0.0000,  3.0000,  0.0000,  0.0000]]], grad_fn=<SelectBackward0>)
tensor([[[ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  1.0000,  0.0000,  6.0000,  4.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  2.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000]],

        [[ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  1.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000]],

        [[ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  2.0000,  0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  2.0000,  0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  1.0000,  0.0000,  1.0000,  0.0000,  0.0000],
         [ 0.0000,  2.0000,  0.0000,  0.0000,  0.0000,  0.0000],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000]],
```

```
         [[ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  1.0000,  3.0000,  6.0000,  4.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000]],

         [[ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  1.0000,  1.0000,  4.0000,  1.0000,  0.0000],
          [ 0.0000,  3.0000,  5.0000,  7.0000,  2.0000,  0.0000],
          [ 0.0000,  2.0000,  2.0000,  1.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000]],

         [[ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  1.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  1.0000,  1.0000,  4.0000,  3.0000,  0.0000],
          [ 0.0000,  0.0000,  1.0000,  1.0000,  1.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000]],

         [[ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  1.0000,  1.0000,  1.0000,  2.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  1.0000,  1.0000,  1.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000]],

         [[ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  3.0000,  0.0000,  2.0000,  0.0000],
          [ 0.0000,  4.0000, 11.0000,  3.0000,  3.0000,  0.0000],
          [ 0.0000,  5.0000, 11.0000,  5.0000,  3.0000,  0.0000],
          [ 0.0000,  0.0000,  3.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000]]],
       grad_fn=<CopySlices>)
torch.Size([8, 6, 6])
tensor([[ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
        [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
        [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
        [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
        [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
        [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
        [ 1.0000,  0.0000,  2.0000,  1.0000,  1.0000,  1.0000,  1.0000,  0.000
0],
        [ 0.0000,  0.0000,  0.0000,  3.0000,  1.0000,  0.0000,  1.0000,  3.000
0],
        [ 6.0000,  0.0000,  0.0000,  6.0000,  4.0000,  0.0000,  1.0000,  0.000
0],
        [ 4.0000,  0.0000,  0.0000,  4.0000,  1.0000,  0.0000,  2.0000,  2.000
0],
        [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
```

```
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
         [ 0.0000,  0.0000,  2.0000,  0.0000,  3.0000,  0.0000,  0.0000,  4.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  5.0000,  0.0000,  0.0000, 11.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  7.0000,  0.0000,  0.0000,  3.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  2.0000,  0.0000,  0.0000,  3.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
         [ 0.0000,  0.0000,  1.0000,  0.0000,  2.0000,  1.0000,  0.0000,  5.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  2.0000,  1.0000,  0.0000, 11.000
0],
         [ 0.0000,  0.0000,  1.0000,  0.0000,  1.0000,  4.0000,  0.0000,  5.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  3.0000,  0.0000,  3.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
         [ 0.0000,  0.0000,  2.0000,  0.0000,  0.0000,  0.0000,  1.0000,  0.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  1.0000,  1.0000,  3.000
0],
         [ 2.0000,  1.0000,  0.0000,  0.0000,  0.0000,  1.0000,  1.0000,  0.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  1.0000,  0.0000,  0.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0],
         [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.000
0]],
       grad_fn=<PermuteBackward0>)
torch.Size([36, 8])
tensor([[[-7.0000, -7.0000, -7.0000, -7.0000,  5.0000,  7.0000,  7.0000,
           1.0000, -3.0000],
         [ 0.0000, -7.0000, -7.0000,  7.0000,  2.0000,  7.0000,  0.0000,
           3.0000,  0.0000],
         [ 1.0000,  4.0000, -7.0000, -2.0000, -4.0000, -5.0000, -1.0000,
          -2.0000, -7.0000],
         [-2.0000, -2.0000,  7.0000, -7.0000, -7.0000, -3.0000, -3.0000,
          -1.0000, -7.0000],
         [ 7.0000,  7.0000, -2.0000,  3.0000,  7.0000, -0.0000, -7.0000,
           5.0000, -2.0000],
```

```
 [-5.0000, -4.0000,  4.0000, -2.0000,  1.0000,  4.0000,  7.0000,
   7.0000,  7.0000],
 [-7.0000, -7.0000, -2.0000,  2.0000,  3.0000,  7.0000,  4.0000,
   4.0000,  2.0000],
 [ 7.0000,  4.0000,  2.0000,  3.0000,  6.0000,  7.0000,  7.0000,
  -2.0000,  4.0000]],

[[-1.0000,  6.0000,  7.0000,  2.0000,  7.0000,  7.0000, -6.0000,
  -4.0000,  2.0000],
 [ 2.0000,  4.0000, -5.0000, -7.0000,  7.0000, -6.0000, -7.0000,
   2.0000, -2.0000],
 [-1.0000,  2.0000, -3.0000,  5.0000, -0.0000,  1.0000, -1.0000,
  -5.0000, -7.0000],
 [-2.0000,  0.0000,  0.0000,  7.0000,  2.0000, -3.0000, -0.0000,
  -7.0000, -7.0000],
 [ 5.0000,  6.0000,  7.0000,  5.0000,  2.0000,  7.0000,  4.0000,
   2.0000,  5.0000],
 [ 4.0000,  7.0000,  1.0000,  1.0000,  7.0000,  7.0000, -7.0000,
  -7.0000, -7.0000],
 [-7.0000,  7.0000,  7.0000, -7.0000, -2.0000,  5.0000, -7.0000,
  -7.0000, -7.0000],
 [-7.0000, -3.0000, -5.0000,  1.0000, -3.0000,  2.0000,  4.0000,
   2.0000,  1.0000]],

[[-0.0000,  2.0000,  5.0000,  7.0000,  5.0000,  7.0000, -1.0000,
  -5.0000, -7.0000],
 [ 7.0000, -6.0000, -7.0000,  4.0000, -7.0000, -7.0000,  7.0000,
  -0.0000,  1.0000],
 [-1.0000,  2.0000,  7.0000, -1.0000, -2.0000, -0.0000, -4.0000,
  -5.0000, -7.0000],
 [-7.0000, -7.0000, -2.0000,  7.0000,  2.0000,  7.0000,  7.0000,
   7.0000,  7.0000],
 [ 2.0000, -1.0000,  3.0000, -5.0000,  4.0000,  7.0000, -5.0000,
  -1.0000,  0.0000],
 [ 0.0000, -3.0000,  0.0000, -7.0000, -4.0000,  0.0000, -7.0000,
  -3.0000,  3.0000],
 [ 2.0000,  2.0000,  0.0000, -2.0000,  3.0000, -1.0000, -2.0000,
  -1.0000, -1.0000],
 [ 4.0000,  7.0000, -1.0000,  1.0000,  7.0000,  4.0000, -2.0000,
   7.0000, -5.0000]],

[[-3.0000,  2.0000, -4.0000, -1.0000, -1.0000, -7.0000,  7.0000,
  -7.0000, -7.0000],
 [-1.0000, -3.0000, -3.0000,  7.0000,  2.0000, -1.0000, -4.0000,
  -0.0000, -0.0000],
 [-1.0000,  7.0000,  7.0000, -2.0000,  7.0000,  7.0000, -7.0000,
  -7.0000,  1.0000],
 [-1.0000,  1.0000,  1.0000, -6.0000,  1.0000,  7.0000,  0.0000,
  -4.0000, -5.0000],
 [ 7.0000,  7.0000, -5.0000,  7.0000,  1.0000,  7.0000, -7.0000,
  -3.0000, -2.0000],
 [-3.0000, -4.0000,  7.0000,  1.0000,  5.0000,  7.0000, -4.0000,
   2.0000,  7.0000],
 [ 3.0000,  1.0000, -7.0000,  7.0000,  4.0000,  2.0000, -3.0000,
   3.0000, -7.0000],
 [-7.0000, -7.0000, -1.0000, -7.0000, -3.0000, -1.0000, -7.0000,
  -2.0000, -7.0000]],

[[ 7.0000,  7.0000,  7.0000,  7.0000,  7.0000,  7.0000, -7.0000,
  -0.0000, -1.0000],
```

```
 [ 1.0000,  1.0000,  4.0000,  7.0000,  2.0000,  4.0000, -1.0000,
  -1.0000, -2.0000],
 [-5.0000,  7.0000,  7.0000,  6.0000,  4.0000,  4.0000, -2.0000,
  -7.0000, -7.0000],
 [ 4.0000,  1.0000,  1.0000, -3.0000,  0.0000,  4.0000, -7.0000,
  -3.0000,  6.0000],
 [-2.0000,  7.0000,  7.0000, -3.0000, -2.0000, -7.0000, -6.0000,
  -7.0000, -7.0000],
 [-2.0000,  3.0000, -1.0000,  5.0000, -7.0000, -7.0000,  7.0000,
   1.0000, -7.0000],
 [-7.0000, -2.0000, -7.0000, -1.0000, -2.0000, -7.0000, -4.0000,
   1.0000,  4.0000],
 [-4.0000,  1.0000, -3.0000, -5.0000, -0.0000, -7.0000, -5.0000,
   5.0000, -3.0000]],

[[-4.0000, -5.0000,  2.0000, -7.0000,  3.0000, -1.0000,  7.0000,
   4.0000,  5.0000],
 [ 7.0000,  7.0000,  7.0000, -0.0000,  7.0000,  7.0000, -7.0000,
  -3.0000,  1.0000],
 [-2.0000, -5.0000, -2.0000, -1.0000, -1.0000, -1.0000, -2.0000,
  -2.0000, -2.0000],
 [ 7.0000, -2.0000,  7.0000,  7.0000,  5.0000,  6.0000,  7.0000,
  -7.0000, -0.0000],
 [ 5.0000,  3.0000, -7.0000,  2.0000,  7.0000,  7.0000, -2.0000,
   7.0000,  7.0000],
 [ 7.0000, -4.0000,  7.0000, -7.0000, -7.0000, -3.0000,  4.0000,
  -2.0000,  1.0000],
 [-2.0000,  2.0000, -1.0000,  4.0000,  3.0000,  0.0000,  3.0000,
  -2.0000, -7.0000],
 [-7.0000, -0.0000, -5.0000,  3.0000, -0.0000,  4.0000,  5.0000,
  -5.0000, -4.0000]],

[[ 0.0000,  7.0000,  0.0000, -7.0000, -3.0000, -5.0000, -7.0000,
  -1.0000, -2.0000],
 [ 7.0000,  0.0000, -0.0000, -1.0000, -1.0000, -4.0000,  7.0000,
   2.0000, -0.0000],
 [ 7.0000, -7.0000, -7.0000,  7.0000,  7.0000,  1.0000,  7.0000,
   7.0000,  4.0000],
 [-7.0000, -2.0000,  7.0000, -5.0000,  1.0000, -5.0000,  0.0000,
   7.0000, -0.0000],
 [ 7.0000,  7.0000, -4.0000,  5.0000,  7.0000, -0.0000, -4.0000,
  -1.0000, -7.0000],
 [-3.0000, -5.0000, -7.0000, -4.0000, -5.0000, -7.0000, -1.0000,
  -3.0000, -2.0000],
 [ 1.0000,  7.0000,  7.0000,  1.0000,  7.0000,  3.0000,  4.0000,
   1.0000, -7.0000],
 [-7.0000,  4.0000, -1.0000, -4.0000,  5.0000,  6.0000,  1.0000,
   5.0000, -1.0000]],

[[-7.0000, -4.0000, -1.0000, -7.0000, -1.0000,  7.0000, -7.0000,
   1.0000,  4.0000],
 [ 7.0000,  7.0000,  7.0000, -0.0000, -2.0000,  1.0000, -5.0000,
  -7.0000,  7.0000],
 [-2.0000, -1.0000, -0.0000, -0.0000,  2.0000,  7.0000,  7.0000,
   4.0000,  7.0000],
 [-4.0000, -1.0000,  7.0000,  7.0000, -4.0000, -4.0000,  7.0000,
   1.0000, -7.0000],
 [ 7.0000, -2.0000, -7.0000,  7.0000, -1.0000, -7.0000,  3.0000,
  -3.0000, -7.0000],
 [ 6.0000, -2.0000,  1.0000,  5.0000,  7.0000,  2.0000,  2.0000,
```

```
          -5.0000, -4.0000],
         [-7.0000, -3.0000, -2.0000, -7.0000,  1.0000,  3.0000,  2.0000,
           7.0000,  1.0000],
         [ 2.0000,  1.0000,  7.0000,  1.0000,  4.0000,  3.0000, -2.0000,
          -3.0000, -2.0000]]], grad_fn=<ReshapeAliasBackward0>)
torch.Size([8, 8, 9])
```

Out[45]:

```
tensor([[ 57.0000,  65.0000,   5.0000,   0.0000,   0.0000,  34.0000,  12.0000,
           0.0000],
        [ 29.0000, 168.0000, 169.0000,   0.0000,   0.0000, 107.0000,   0.0000,
           0.0000],
        [135.0000, 216.0000,  99.0000,   0.0000,   0.0000, 231.0000,   0.0000,
           0.0000],
        [  0.0000, 145.0000,  92.0000,   0.0000,   0.0000,  57.0000,   0.0000,
          10.0000],
        [184.0000,  96.0000,  79.0000,   0.0000,   0.0000,  36.0000, 142.0000,
           0.0000],
        [184.0000, 153.0000, 192.0000,   0.0000,   0.0000, 114.0000, 223.0000,
           0.0000],
        [240.0000, 144.0000,   9.0000,   0.0000,   0.0000, 128.0000, 142.0000,
           0.0000],
        [ 64.0000,  46.0000,   0.0000,   0.0000,  61.0000,  68.0000,  98.0000,
           0.0000],
        [200.0000,   1.0000, 104.0000,   0.0000,   0.0000,   0.0000,  95.0000,
          92.0000],
        [276.0000,  32.0000, 201.0000,   0.0000,   0.0000,   0.0000, 133.0000,
         115.0000],
        [337.0000,  15.0000,  72.0000,   0.0000,   0.0000,   0.0000,   0.0000,
         138.0000],
        [157.0000,   8.0000,  18.0000,   6.0000,   0.0000,   0.0000,  23.0000,
          99.0000],
        [ 83.0000,   0.0000,  37.0000,   0.0000,   0.0000,   0.0000,  31.0000,
          81.0000],
        [168.0000,   0.0000, 122.0000,   0.0000,   0.0000,   0.0000,   6.0000,
          98.0000],
        [148.0000,   0.0000,  65.0000,   0.0000,   0.0000,   0.0000,   0.0000,
          67.0000],
        [ 17.0000,   0.0000,  38.0000,   0.0000,   0.0000,   0.0000,   0.0000,
          27.0000]], grad_fn=<AsStridedBackward0>)
```

In [46]:

```
print(a_pad_nij.shape)
print(w_int_kij.shape)
print(out_int_nij.shape)
```

```
torch.Size([36, 8])
torch.Size([8, 8, 9])
torch.Size([16, 8])
```

In [43]:

```
bit_precision = 4
file = open('activation.txt', 'w') #write to file
file.write('#time0row7[msb-lsb],time0row6[msb-lst],....,time0row0[msb-lst]#\n'
file.write('#time1row7[msb-lsb],time1row6[msb-lst],....,time1row0[msb-lst]#\n'
file.write('#................#\n')
for nij in range(a_pad_nij.shape[0]):
    for row in range(a_pad_nij.shape[1] -1, -1, -1): # get the last row
        X_bin = '{0:04b}'.format(round(a_pad_nij[nij][row].item()))
        for k in range(bit_precision):
            file.write(X_bin[k])

    file.write("\n")
file.close()
```

In [47]:
```
bit_precision = 16
file = open('out.txt', 'w') #write to file
file.write('#time0row7[msb-lsb],time0row6[msb-lst],....,time0row0[msb-lst]#\n'
file.write('#time1row7[msb-lsb],time1row6[msb-lst],....,time1row0[msb-lst]#\n'
file.write('#................#\n')
for nij in range(out_int_nij.shape[0]):
    for row in range(out_int_nij.shape[1] -1, -1, -1): # get the last row
        X_bin = '{0:016b}'.format(round(out_int_nij[nij][row].item()))
        for k in range(bit_precision):
            file.write(X_bin[k])

    file.write("\n")
file.close()
```

In [40]:
```
# returns bit string of pos number with bit width bw
def bit_string_pos(num, bw):
    bin_rep = bin(num)[2:]
    rem_zeros = bw - len(bin_rep)
    return ("0" * rem_zeros) + bin_rep

bit_precision = 4

### Complete this cell ###
for kij in range(9):
    file = open(f'weight_k{kij+1}.txt', 'w')
    file.write('#col0row7[msb-lsb],col0row6[msb-lst],....,col0row0[msb-lst]#\n
    file.write('#col1row7[msb-lsb],col1row6[msb-lst],....,col1row0[msb-lst]#\n
    file.write('#................#\n')

    for col in range(w_int_kij.shape[1]):
        for row in range(w_int_kij.shape[0] - 1, -1, -1):
            val = int(w_int_kij[col][row][kij])
            if val >= 0: # positive
                file.write("0")
                file.write(bit_string_pos(val, bit_precision-1))
            else: # negative
                file.write("1")
                pos_val = ((~abs(val)) & (2 ** (bit_precision-1) - 1)) + 1
                file.write(bit_string_pos(pos_val, bit_precision-1))
        file.write("\n")
    file.close()
```

In [ ]: